

SUJET 07 — DEVSECOPS BAC+4

Infrastructure as Code Security

Comment sécuriser son infrastructure Terraform avant même de la déployer

Formation : DevSecOps BAC+4 — Jedy Formation

Formateur : Kalidou DIA

Repo GitHub : varshanroshan/Infrastructure-as-Code-Security

Année : 2025 – 2026

Terraform

Checkov SAST

GitHub Actions

AWS CIS Benchmark

Sommaire

1. Introduction — L'histoire qui a tout changé	
2. L'Infrastructure as Code : puissante, mais risquée	
3. Quel outil choisir ? Notre comparaison honnête	
4. Notre démo — Ce qu'on a fait concrètement	4.1 Le code volontairement cassé
	4.2 Les 35 failles détectées
	4.3 La correction faille par faille
5. Le pipeline GitHub Actions — Le vigile automatique	
6. Le principe du moindre privilège	
7. Ce qu'on a appris et les limites de l'outil	
8. Conclusion	
Annexes	A. Code complet vulnerable/main.tf
	B. Code complet secure/main.tf
	C. Workflow GitHub Actions
	D. Liste des 35 failles Checkov

1 Introduction — L'histoire qui a tout changé

Capital One, 2019 : une ligne de code, 100 millions de victimes

En mars 2019, une ingénieure du nom de Paige Thompson quitte Amazon Web Services après plusieurs années à travailler sur l'infrastructure cloud. Elle connaît le système par cœur. Quelques mois plus tard, elle remarque quelque chose d'étrange dans la configuration publique d'un serveur appartenant à Capital One, l'une des plus grandes banques américaines.

Le problème ? Un rôle IAM (c'est-à-dire une permission d'accès configurée dans le cloud AWS) était mal configuré dans le code Terraform de la banque. Ce rôle autorisait n'importe quelle machine à lui demander des identifiants. En quelques requêtes HTTP simples, Paige Thompson a pu récupérer des clés d'accès temporaires. Avec ces clés, elle a accédé à plus de 700 dossiers S3 (des espaces de stockage dans le cloud). À l'intérieur : les données personnelles de **106 millions de clients**.

"La vulnérabilité n'était pas dans un algorithme complexe. Elle était dans une configuration. Quelqu'un avait écrit une règle de sécurité trop permissive, et personne ne l'avait détectée avant le déploiement."

Capital One a payé 80 millions de dollars d'amende. La réputation de la banque en a pris un coup immense. Et le plus douloureux dans cette histoire ? La faille aurait pu être détectée **avant même que le code soit déployé**, avec des outils qui existaient déjà à l'époque.

C'est exactement ce problème que nous avons voulu explorer dans ce projet. Comment on évite ça ? Comment on fait pour qu'une erreur de configuration soit détectée dès l'écriture du code, et pas des mois plus tard par une ancienne employée malveillante ?

Notre question de départ

Au début de ce projet, notre groupe partait d'une question simple : est-ce qu'il est possible d'automatiser la détection des erreurs de sécurité dans du code d'infrastructure, de la même façon qu'on automatise les tests unitaires pour du code applicatif ?

La réponse, on l'a découverte assez vite, est oui. Et c'est là que le concept de "shift-left" entre en jeu. Shift-left, ça veut dire littéralement "décaler vers la gauche" sur la frise chronologique d'un projet. En sécurité, ça signifie : **détecter les problèmes le plus tôt possible dans le cycle de développement**, plutôt que d'attendre les tests en production ou, pire, un incident réel.

Notre sujet 07 nous a donc amenés à construire une démo complète : un fichier Terraform volontairement truffé de failles, un scanner qui les détecte automatiquement, une version corrigée du même fichier, et un pipeline qui vérifie tout ça à chaque fois qu'on pousse du code sur GitHub.

35

FAILLES DÉTECTÉES DANS
LE CODE VULNÉRABLE

4

FAMILLES DE
VULNÉRABILITÉS SIMULÉES

3

JOBS DANS LE PIPELINE
CI/CD

2 L'Infrastructure as Code : puissante, mais risquée

C'est quoi l'IaC ?

Avant de commencer le projet, on avait une idée vague de ce qu'était l'IaC. On pensait que c'était juste du YAML ou du JSON pour décrire des serveurs. En fait, c'est bien plus que ça.

L'Infrastructure as Code (IaC), c'est l'idée de décrire toute son infrastructure cloud sous forme de fichiers texte. Plutôt que de cliquer dans des interfaces graphiques pour créer un serveur, un bucket de stockage ou un pare-feu, on écrit du code. Ce code est ensuite exécuté par un outil comme Terraform, qui crée ou modifie les ressources dans le cloud en suivant les instructions à la lettre.

Imaginez que vous construisez des maisons. Sans IaC, chaque maison est construite à la main par des ouvriers qui font des choix au fur et à mesure. Avec IaC, vous avez **un plan architectural écrit en code**. L'outil lit ce plan et construit la maison exactement comme décrit. La maison sera identique à chaque fois qu'on repart du même plan. C'est reproductible, versionnable avec Git, et auditable.

TERRAFORM EN DEUX MOTS

Terraform est l'outil IaC le plus populaire, créé par HashiCorp. On écrit des fichiers `.tf` qui décrivent des ressources AWS, Azure ou Google Cloud. Terraform calcule les changements nécessaires et les applique. C'est comme un "git pour l'infrastructure".

Le problème : si le plan est mauvais, toutes les maisons sont mauvaises

La grande force de l'IaC devient aussi sa grande faiblesse. Si le plan architectural dit "laisse la porte d'entrée ouverte à tout le monde", alors **toutes les maisons construites à partir de ce plan auront la porte ouverte**. Et si ce plan est utilisé pour créer 50 environnements (développement, test, staging, production...), vous avez 50 portes ouvertes.

Dans le monde cloud, ça se traduit par des erreurs de configuration qui se répliquent à grande échelle. Une mauvaise règle IAM se retrouve dans tous les environnements. Un bucket S3 mal configuré est déployé partout. Et contrairement à un bug dans du code applicatif, une erreur de configuration peut ne générer aucune erreur apparente : tout fonctionne, mais la sécurité est absente.

Les 4 risques qu'on a simulés dans notre projet

Risque 1 — Le secret hardcodé (comme la clé sous le paillasson)

Dans notre fichier `vulnerable/main.tf`, on a écrit le mot de passe de la base de données directement dans le code :

```
password = "SuperPassword123!" # DANGER : secret hardcodé !
```

C'est exactement comme laisser sa clé sous le paillasson devant sa porte. N'importe qui qui a accès au dépôt Git — même en lecture seule — peut lire ce mot de passe. Pire : ce mot de passe se retrouve dans l'historique Git, dans le fichier d'état Terraform (`terraform.tfstate`), et potentiellement dans les logs CI/CD. Même si on corrige ça plus tard, la trace reste.

Risque 2 — Les permissions trop larges (le passe-partout de l'immeuble)

On a créé une politique IAM avec `Action = "*" et Resource = "*" . En pratique, ça revient à donner à une entité (un utilisateur, un service) les droits d'administrateur complet sur tout le compte AWS. C'est comme donner le passe-partout de tout un immeuble à quelqu'un qui n'a besoin que d'accéder au couloir du rez-de-chaussée. Si cette entité est compromise, c'est toute l'infrastructure AWS qui tombe.`

Risque 3 — Le bucket S3 public (ses documents confidentiels sur Internet)

AWS S3 est un service de stockage dans le cloud. Par défaut, les buckets sont privés. Mais en ajoutant `acl = "public-read"`, on rend tout le contenu du bucket accessible à n'importe qui sur Internet, sans authentification. Imaginez publier vos documents RH, vos sauvegardes de base de données, ou vos logs sur Internet. C'est exactement ce que fait cette configuration.

Risque 4 — Le configuration drift (le plan qui ne correspond plus à la réalité)

Le drift (en français : dérive), c'est quand la configuration réelle de l'infrastructure ne correspond plus au code IaC. Quelqu'un a modifié un pare-feu à la main via la console AWS. Une règle a été ajoutée manuellement en urgence. Le code dit une chose, la réalité en est une autre. Ce gap est dangereux car les outils de sécurité analysent le code, pas la réalité. Une faille introduite manuellement passe sous les radars.

LE DRIFT EST INVISIBLE

Checkov et les outils similaires analysent le code IaC. Si quelqu'un ouvre un port de sécurité directement dans AWS sans modifier le code Terraform, aucun outil d'analyse statique ne le détectera. C'est une limite importante qu'on détaille dans la section 7.

3 Quel outil choisir ? Notre comparaison honnête

Avant de commencer à coder, on s'est retrouvés face à un problème classique : il existe plusieurs outils pour analyser la sécurité du code laC, et on ne savait pas lequel choisir. Franchement, au début on ne connaissait aucun de ces noms. On a passé quelques heures à lire la documentation et à faire des tests rapides.

Les 4 candidats qu'on a étudiés

Outil	Créé par	Langages laC supportés	Licence	Notre verdict
Checkov	Prisma Cloud (Palo Alto)	Terraform, CloudFormation, K8s, Dockerfile, ARM...	Open source (Apache 2.0)	Notre choix ✓
tfsec	Aqua Security	Terraform uniquement	Open source (MIT)	Bon mais limité
Terrascan	Tenable	Terraform, K8s, Dockerfile, Helm...	Open source (Apache 2.0)	Bien, moins documenté
KICS	Checkmarx	Terraform, CloudFormation, K8s, Docker, Ansible...	Open source (Apache 2.0)	Complet mais lourd

Pourquoi on a choisi Checkov

Plusieurs raisons ont guidé notre choix. D'abord, la **facilité d'installation** : un simple `pip install checkov` et c'est opérationnel. Pas besoin de Docker, pas besoin de compiler quoi que ce soit. Pour un projet étudiant avec des deadlines, c'est important.

Ensuite, la **richesse des règles** : Checkov embarque plus de 1000 règles de sécurité basées sur les benchmarks CIS (Center for Internet Security), le framework AWS Well-Architected, et les standards SOC2. Sur notre seul fichier Terraform de 170 lignes, il a trouvé 35 problèmes.

La **lisibilité des rapports** nous a aussi convaincus. Chaque faille est identifiée par un code unique (comme `CKV_AWS_20`), accompagnée d'une explication et d'un lien vers la documentation. On comprend tout de suite pourquoi c'est problématique et comment corriger.

Enfin, l'**intégration GitHub Actions** est native. On peut lancer Checkov en une ligne de commande dans un workflow YAML, récupérer un rapport JSON, et bloquer un push si des failles critiques sont

détectées.

Ce qu'on a trouvé moins bien chez Checkov

Soyons honnêtes : Checkov n'est pas parfait. Il génère parfois des **faux positifs** (c'est-à-dire des alertes sur des choses qui ne sont pas réellement des problèmes dans notre contexte). Sur notre fichier `secure/main.tf`, quelques règles concernant la réplication cross-région d'un bucket S3 ou les notifications d'événements ont déclenché des alertes, alors que ces fonctionnalités ne sont pas indispensables dans un projet de démo.

Il ne détecte aussi **que les problèmes dans le code écrit**. Si quelqu'un modifie la configuration directement dans AWS sans toucher au code Terraform, Checkov n'en saura rien. Pour ça, il faudrait un outil de type AWS Config ou Drift Detection, ce qui dépasse le cadre de notre projet.

Et tfsec ?

On a quand même testé tfsec en parallèle. Il est plus rapide et produit moins de bruit (moins de faux positifs). Mais il se concentre uniquement sur Terraform. Or, dans un contexte professionnel réel, on travaille souvent avec plusieurs technologies : Kubernetes pour les conteneurs, Dockerfile pour les images, CloudFormation si on est sur un stack AWS pur... Checkov couvre tout ça, ce qui justifie notre choix pour un projet qui se veut représentatif d'un environnement DevSecOps complet.

4 Notre démo — Ce qu'on a fait concrètement

4.1 — On a commencé par casser volontairement notre infrastructure

L'idée de départ était de créer un fichier Terraform qui représente une infrastructure AWS "classique" : un bucket de stockage S3, un groupe de sécurité réseau, une base de données RDS, et une politique de droits IAM. Puis on a volontairement introduit des failles connues dans chaque ressource.

Ce n'était pas si simple que ça en avait l'air. On pensait au début introduire 4 ou 5 failles et se retrouver avec autant d'alertes Checkov. En réalité, chaque mauvaise pratique déclenche souvent **plusieurs règles** en cascade. Par exemple, le fait de ne pas chiffrer un bucket S3 viole à la fois la règle `CKV_AWS_19` (chiffrement au repos) et `CKV_AWS_145` (chiffrement via KMS). Résultat : on a obtenu 35 alertes sur un seul fichier de moins de 170 lignes.

⚠ AVERTISSEMENT IMPORTANT

Le fichier `vulnerable/main.tf` contient des vulnérabilités intentionnelles à des fins pédagogiques. Il ne doit jamais être déployé sur un compte AWS réel. Les mots de passe en clair sont fictifs.

4.2 — Les 35 failles détectées : on ne s'y attendait pas

On a lancé Checkov avec cette commande simple :

```
checkov -f vulnerable/main.tf --compact
```

Le résultat est tombé en quelques secondes :

```
Passed checks: 13
Failed checks: 35
Skipped checks: 0
```

Franchement, on ne s'attendait pas à autant. 35 failles sur un seul fichier. Et ça, en quelques secondes, sans déployer quoi que ce soit sur AWS. C'est ça la puissance de l'analyse statique : elle fonctionne sur le code source, pas sur l'infrastructure réelle.

4.3 — Les 4 failles principales expliquées une par une

Faillle 1 — S3 public (CKV_AWS_20)

✗ CODE VULNÉRABLE

```
resource "aws_s3_bucket_acl" "vuln" {
  bucket = aws_s3_bucket.bucket.id
  acl    = "public-read"
  # Tout internet peut lire ce bucket
}
```

✓ CODE SÉCURISÉ

```
resource "aws_s3_bucket_public_access_block" " "
  bucket = aws_s3_bucket.bucket.id
  block_public_acls      = true
  block_public_policy    = true
  ignore_public_acls    = true
  restrict_public_buckets = true
}
```

La correction ne supprime pas juste l'ACL publique. Elle active le mécanisme `public_access_block` d'AWS, qui est un verrou au niveau du compte : même si quelqu'un essaie d'ajouter une règle publique plus tard (par erreur ou par malice), ce verrou l'empêchera de prendre effet.

Faillie 2 — Security Group ouvert (CKV_AWS_24 + CKV_AWS_25)

✗ CODE VULNÉRABLE

```
ingress {
  from_port = 22
  to_port   = 22
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  # SSH ouvert au monde entier
}
ingress {
  from_port = 0
  to_port   = 65535
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}
```

✓ CODE SÉCURISÉ

```
ingress {
  from_port = 80
  to_port   = 80
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  description = "HTTP → redirige HTTPS"
}
ingress {
  from_port = 443
  to_port   = 443
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  description = "HTTPS depuis internet"
}
```

Un Security Group, c'est le pare-feu d'AWS. Dans le code vulnérable, on ouvre les ports 22 (SSH) et 3389 (RDP) à tout internet (`0.0.0.0/0` signifie "n'importe quelle adresse IP"). N'importe qui peut donc tenter de se connecter à nos serveurs. Dans la version corrigée, seuls les ports HTTP (80) et HTTPS (443) sont ouverts depuis internet. Le SSH n'est accessible que via un réseau privé ou un bastion.

Faillle 3 — RDS non sécurisé (CKV_AWS_16, CKV_AWS_17, CKV_AWS_129)

✗ CODE VULNÉRABLE

```
resource "aws_db_instance" "vuln_rds" {
  username      = "admin"
  password      = "SuperPassword123!"
  storage_encrypted = false
  publicly_accessible = true
  backup_retention_period = 0
  deletion_protection = false
  multi_az      = false
}
```

✓ CODE SÉCURISÉ

```
resource "aws_db_instance" "secure_rds" {
  username      = "dbadmin"
  password      = var.db_password
  storage_encrypted = true
  publicly_accessible = false
  backup_retention_period = 7
  deletion_protection = true
  multi_az      = true
}
```

Cette ressource RDS (Relational Database Service — c'est-à-dire une base de données MySQL dans le cloud) accumule les problèmes. Le mot de passe est en clair dans le code. La base est accessible depuis internet. Les données ne sont pas chiffrées. Il n'y a pas de backup automatique. Et si quelqu'un supprime accidentellement cette base de données, il n'y a aucune protection. La version sécurisée corrige tous ces points en quelques lignes.

Faillle 4 — IAM trop permissif (CKV_AWS_62, CKV_AWS_40)

✗ CODE VULNÉRABLE

```
policy = jsonencode({
  Statement = [{
    Effect   = "Allow"
    Action  = "*"
    Resource = "*"
  }]
})
# Droits admin complets sur tout AWS
```

✓ CODE SÉCURISÉ

```
policy = jsonencode({
  Statement = [{
    Effect = "Allow"
    Action = [
      "s3:GetObject",
      "s3:ListBucket"
    ]
    Resource = [
      aws_s3_bucket.bucket.arn
    ]
  }]
})
```

C'est probablement la faille la plus dangereuse de notre démo. `Action = "*" avec Resource = "*" , c'est l'équivalent d'un accès administrateur complet sur l'intégralité du compte AWS. Si cette entité est compromise, l'attaquant peut créer des ressources, supprimer des données, exfiltrer des secrets, créer de nouveaux utilisateurs. La version corrigée limite les droits à deux actions précises (s3:GetObject et s3:ListBucket) sur un seul bucket spécifique.`

Ce qui nous a le plus surpris

Honnêtement, ce qui nous a surpris c'est la rapidité du scan. En moins de 5 secondes, Checkov avait analysé tout notre fichier et identifié 35 problèmes. Chaque problème était accompagné de son identifiant unique, d'une description claire, et d'un lien vers la documentation pour comprendre comment le corriger. On n'avait jamais vu un outil aussi direct et actionnable.

On a aussi été surpris par le fait que certaines règles qu'on pensait "triviales" ou "cosmétiques" ont une vraie importance de sécurité. Par exemple, `CKV_AWS_23` demande que chaque règle de Security Group ait une description. Au premier abord, ça semble accessoire. Mais sans description, impossible pour une équipe de sécurité de savoir *pourquoi* tel port est ouvert, ni si c'est intentionnel ou une erreur oubliée.

5 Le pipeline GitHub Actions — Le vigile automatique

L'idée en une image

Imaginez l'entrée d'un immeuble sécurisé. Il y a un vigile. Chaque personne qui veut entrer passe devant lui. Si elle a un badge valide, elle entre. Sinon, elle est bloquée. Maintenant imaginez que ce vigile fonctionne 24h/24, 7j/7, sans jamais se fatiguer, et qu'il vérifie non seulement les badges mais aussi si la personne transporte des objets interdits.

C'est exactement ce que fait notre pipeline GitHub Actions. Chaque fois qu'un membre de l'équipe pousse du code sur GitHub, le "vigile" se réveille, analyse le code Terraform, et décide si le code peut continuer son chemin vers le déploiement.



Les 3 jobs du workflow

Notre pipeline contient trois jobs qui s'exécutent en parallèle (sauf le troisième qui attend les deux premiers).

Job 1 — Scan du code vulnérable (non bloquant)

Ce job scanne le répertoire `vulnerable/`. Les failles détectées ici sont intentionnelles — on ne veut pas que le pipeline s'arrête à cause d'elles. On utilise donc `|| true` pour que la commande retourne toujours succès, même si Checkov trouve des problèmes. C'est purement informatif, pour montrer combien de failles existent dans le code non sécurisé.

Job 2 — Scan du code sécurisé (bloquant)

Ce job scanne le répertoire `secure/`. Ici, si Checkov détecte une faille, le pipeline s'arrête avec une erreur. C'est le "gate qualité" : si quelqu'un modifie le code sécurisé et introduit une régression, la PR est bloquée. On ne peut pas fusionner tant que toutes les règles Checkov ne passent pas.

Job 3 — Résumé comparatif

Ce job s'exécute après les deux premiers (même s'ils échouent, grâce à `if: always()`). Il génère un rapport comparatif qui montre côte à côte les résultats des deux scans. Les rapports JSON sont uploadés en tant qu'artifacts GitHub (des fichiers attachés au run) et conservés pendant 30 jours.

Le fichier YAML expliqué ligne par ligne

```
name: IaC Security - Checkov SAST Scan

on:
    # Quand le pipeline se déclenche
    push:
        branches: [main]          # Sur chaque push vers main
    pull_request:
        branches: [main]         # Sur chaque PR vers main

jobs:
    checkov-vulnerable:
        runs-on: ubuntu-latest    # Machine Linux gratuite GitHub
        steps:
            - uses: actions/checkout@v4 # Télécharge le code
            - uses: actions/setup-python@v5
              with:
                python-version: "3.11"
            - run: pip install checkov # Installe Checkov
            - run: |
                checkov -d vulnerable/ \
                  --compact \
                  --output json \
                  --output-file-path vulnerable-report \
                  || true # NE BLOQUE PAS (vulns intentionnelles)

    checkov-secure:
        runs-on: ubuntu-latest
        steps:
            # [setup identique...]
            - run: |
                checkov -d secure/ \
                  --compact \
                  --output json \
                  --output-file-path secure-report
                # PAS de || true : bloque si faille détectée
```

La différence entre les deux scans, c'est juste `|| true`

Ce petit détail est au cœur de notre stratégie. En bash, `|| true` signifie "si la commande précédente échoue, retourne quand même succès". C'est ce qui permet au pipeline de continuer même quand Checkov trouve des problèmes dans le code vulnérable. Sur le code sécurisé, on retire ce `|| true` : toute faille devient bloquante. C'est le principe du "quality gate" en intégration continue.

6 Le principe du moindre privilège

C'est un des concepts les plus importants en sécurité informatique, et c'est aussi l'un de ceux qu'on comprend le mieux avec une analogie concrète.

L'analogie du trousseau de clés

Imaginez un immeuble de bureaux. Il y a plusieurs types de personnes qui y travaillent : des employés, des managers, des agents d'entretien, des visiteurs. Chacun a besoin d'accéder à des zones différentes.

Un agent d'entretien n'a pas besoin des clés du coffre-fort de la direction. Il a besoin des clés des toilettes, des couloirs, et peut-être de la salle de pause. Si on lui donne le passe-partout général pour qu'il soit "plus à l'aise", on crée un risque : si ce trousseau est perdu ou volé, quelqu'un peut accéder à tout l'immeuble.

En sécurité informatique, c'est exactement pareil. Le **principe du moindre privilège** (Least Privilege Principle) dit qu'une entité — que ce soit un utilisateur, un service, ou un programme — ne doit avoir accès qu'aux ressources dont elle a strictement besoin pour remplir sa fonction. Pas plus.

Comment ça se traduit en Terraform avec IAM

IAM (Identity and Access Management) est le système de gestion des droits d'AWS. Avec IAM, on définit qui peut faire quoi sur quelles ressources. Les permissions sont définies via des "politiques" (policies) écrites en JSON.

Dans notre code vulnérable, on a écrit la politique la plus permissive possible :

```
# VERSION DANGEREUSE — Action wildcard
{
  "Statement": [{
    "Effect": "Allow",
    "Action": "*", // TOUS les services AWS
    "Resource": "*" // TOUTES les ressources
  }]
}
```

Le symbole `*` est un wildcard — en informatique, c'est le caractère "joker" qui signifie "tout". `Action = "*" signifie que l'entité peut faire n'importe quelle action sur n'importe quel service AWS : créer des`

instances EC2, supprimer des bases de données, modifier des règles de sécurité, exfiltrer des données S3... C'est l'équivalent du passe-partout général de tout l'immeuble.

Dans le code sécurisé, on applique le principe du moindre privilège :

```
# VERSION SÉCURISÉE — Actions minimales sur ressource précise
{
  "Statement": [
    {
      "Sid":      "S3ReadOnlyAccess",
      "Effect":   "Allow",
      "Action":   ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::iac-security-demo-secure-bucket",
        "arn:aws:s3:::iac-security-demo-secure-bucket/*"
      ]
    },
    {
      "Sid":      "CloudWatchLogsWrite",
      "Effect":   "Allow",
      "Action":   [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:eu-west-1:*:log-group:/app/iac-*"
    }
  ]
}
```

On liste exactement les deux choses que cette entité doit pouvoir faire : lire des fichiers dans un bucket S3 précis, et écrire des logs CloudWatch pour un groupe de logs spécifique. Rien de plus. Si cette entité est compromise, l'attaquant ne peut qu'accéder à ce bucket et à ces logs. Le reste du compte AWS est protégé.

Politiques sur les rôles, pas sur les utilisateurs

Un autre point important : dans le code vulnérable, on attache la politique directement à un utilisateur IAM. Dans le code sécurisé, on l'attache à un **rôle IAM**. La différence, c'est que les rôles sont temporaires et sont assumés par des services (comme EC2 ou Lambda). Si un rôle est compromis, ses droits expirent automatiquement. Un utilisateur IAM avec une clé d'accès permanente, c'est un risque qui dure tant que la clé n'est pas révoquée.

BONNE PRATIQUE IAM RÉSUMÉE

Toujours lister les actions nécessaires explicitement (jamais de `*` en production). Toujours préciser les ressources concernées. Utiliser des rôles IAM plutôt que des utilisateurs. Activer la

rotation des clés d'accès. Faire des revues de droits régulières.

7

Ce qu'on a appris et les limites de l'outil

Ce qui a bien marché

La partie technique s'est déroulée sans grande surprise. Checkov est vraiment facile à prendre en main. L'installation prend trente secondes, le premier scan prend moins d'une minute. Le rapport est clair, les règles sont bien documentées, et les liens vers les guides de correction sont utiles.

Le pipeline GitHub Actions a aussi fonctionné dès le premier essai. La logique avec et sans `|| true` pour différencier les scans bloquants et non-bloquants est simple et efficace. On a trouvé cette approche élégante : le même outil, les mêmes commandes, mais une décision différente sur le comportement en cas d'échec.

On a aussi vraiment apprécié le fait de travailler avec de la vraie infrastructure AWS (même simulée). Comprendre ce que signifie un Security Group, ce qu'est un bucket S3, comment IAM fonctionne... tout ça n'est pas évident au départ quand on n'a jamais travaillé avec AWS. Ce projet nous a forcés à comprendre les concepts avant d'écrire la configuration.

Ce qui était plus difficile que prévu

La partie la plus difficile n'était pas technique. C'était de **comprendre pourquoi** certaines règles Checkov existent. Par exemple, `CKV_AWS_144` vérifie que les buckets S3 ont de la réplication cross-région. Au début, on trouvait ça excessif pour un projet de démo. Puis on a réalisé que dans un contexte de production, si la région AWS Paris tombe en panne, les données sont perdues sans réplication. Ce projet nous a appris à ne pas traiter les règles de sécurité comme des contraintes arbitraires, mais à comprendre le risque derrière chacune d'elles.

On a aussi eu du mal avec les **faux positifs**. Quelques règles Checkov ne s'appliquaient pas à notre contexte mais étaient quand même déclenchées. Gérer ça proprement (avec des skip comments ou des fichiers de configuration Checkov) aurait demandé plus de temps qu'on ne l'avait prévu.

Les limites de Checkov

Il serait malhonnête de présenter Checkov comme une solution complète à tous les problèmes de sécurité IaC. Il a des limites importantes.

Limite 1 : Il n'analyse que le code statique. Checkov lit les fichiers `.tf` et vérifie leur contenu. Il ne sait pas ce qui est réellement déployé sur AWS. Si quelqu'un modifie la configuration d'un Security

Group directement via la console AWS sans toucher au code Terraform, Checkov ne verra rien. Pour détecter ce type de dérive (drift), il faut des outils complémentaires comme AWS Config, Terraform Cloud Sentinel, ou driftctl.

Limite 2 : Il ne comprend pas le contexte métier. Checkov applique des règles génériques. Il ne sait pas que votre bucket S3 contient des images publiques de produits (et doit donc être public) ou des sauvegardes de bases de données (qui doivent être privées). Il faut configurer des exceptions manuellement, ce qui demande une connaissance fine de l'outil.

Limite 3 : La qualité des règles varie. Certaines règles sont très bien calibrées. D'autres sont trop permissives ou trop strictes selon le contexte. Un outil seul ne remplace pas une revue humaine du code par un ingénieur sécurité expérimenté.

Limite 4 : Il ne détecte pas les vulnérabilités logiques. Si la logique métier d'une application crée une faille de sécurité (accès à des données d'un autre utilisateur, par exemple), Checkov n'y verra rien. Ce sont des failles qui demandent des tests de sécurité applicatifs (DAST, pentest), pas de l'analyse statique IaC.

Ce qu'on ferait différemment si on recommençait

Si on recommençait ce projet, on irait plus loin sur deux points. D'abord, on testerait l'**intégration avec Terraform Cloud** pour faire tourner Checkov directement dans le plan Terraform avant l'apply, pas juste dans le CI/CD GitHub. C'est une approche encore plus shift-left.

Ensuite, on ajouterait un outil de **détection de drift** pour montrer la différence entre ce qu'on a dans le code et ce qui est réellement déployé. C'est un vrai angle mort de Checkov qu'on n'a pas eu le temps de traiter.

8 Conclusion

En commençant ce projet, on avait une vision un peu abstraite de ce que voulait dire "sécuriser l'infrastructure". On pensait que c'était réservé à des experts en sécurité avec des années d'expérience, des outils complexes, et des certifications spécialisées.

Trois semaines plus tard, notre point de vue a changé.

La sécurité de l'infrastructure cloud, c'est d'abord une question de configuration. Et la configuration, ça s'analyse. Checkov a mis moins de 5 secondes pour trouver 35 problèmes dans notre fichier. Des problèmes qui, en production, auraient pu exposer des données utilisateurs, permettre une intrusion dans nos systèmes, ou donner à un attaquant les clés d'un compte AWS entier.

Ce qui nous a le plus frappés, c'est la différence entre le coût de la prévention et le coût de l'incident. Un scan Checkov coûte 5 secondes et quelques centimes de temps de calcul dans GitHub Actions. L'incident Capital One a coûté 80 millions de dollars d'amende, plus les dommages réputationnels, plus les recours juridiques. La balance est évidente.

"Security as Code, ce n'est pas une tendance. C'est une réponse concrète à un problème concret : les infrastructures cloud se déploient vite, à grande échelle, et une erreur de configuration se réplique aussi vite que le code lui-même."

Ce projet nous a aussi confirmé quelque chose qu'on entend souvent dans le cursus DevSecOps : la sécurité n'est pas un département séparé, c'est une responsabilité partagée. Un développeur qui écrit du code Terraform peut, avec des outils accessibles et gratuits, détecter et corriger des failles de sécurité avant que son code n'atteigne la production. C'est ça, le DevSecOps en pratique.

On repart de ce projet avec une compétence concrète (configurer un pipeline Checkov + GitHub Actions), une meilleure compréhension de l'écosystème AWS (IAM, S3, RDS, Security Groups), et surtout une façon différente de lire le code : en se demandant toujours, "si quelqu'un exploite cette configuration, que peut-il faire ?"

Et pour répondre à la question posée en introduction : comment Capital One aurait pu éviter la catastrophe de 2019 ? Avec un pipeline exactement comme celui qu'on a construit dans ce projet. Une règle de sécurité manquante, détectée automatiquement, corrigée avant le déploiement. C'est simple. C'est accessible. Et ça fonctionne.

CE PROJET EN CHIFFRES

35 failles détectées par Checkov sur le code vulnérable · 4 familles de vulnérabilités corrigées · 3 jobs dans le pipeline CI/CD · 1 rapport JSON uploadé en artifact · 0 euro de coût (tout open source et gratuit)

Remerciements

Nous remercions notre formateur **Kalidou DIA** pour ses retours et son accompagnement tout au long de ce projet. Les sujets proposés par Jedy Formation ont le mérite de nous confronter à des cas réels plutôt qu'à des exercices artificiels. Le sujet 07 sur l'IaC Security est particulièrement représentatif des défis rencontrés dans l'industrie aujourd'hui.

Annexes

Annexe A — Code complet : vulnerable/main.tf

```
# =====  
# FICHIER VOLONTAIREMENT VULNÉRABLE - usage pédagogique  
# Détecté par Checkov (outil SAST pour Terraform)  
# =====  
  
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "eu-west-1"  
}  
  
# VULNÉRABILITÉ 1 - CKV_AWS_20 : Bucket S3 public  
resource "aws_s3_bucket" "vulnerable_bucket" {  
  bucket = "my-super-vulnerable-bucket-demo"  
  tags   = { Name = "VulnerableBucket" }  
}  
  
resource "aws_s3_bucket_acl" "vulnerable_acl" {  
  bucket = aws_s3_bucket.vulnerable_bucket.id  
  acl    = "public-read" # DANGER  
}  
  
resource "aws_s3_bucket_versioning" "vulnerable_versioning" {  
  bucket = aws_s3_bucket.vulnerable_bucket.id  
  versioning_configuration {  
    status = "Disabled" # VULN CKV_AWS_52  
  }  
}  
  
# VULNÉRABILITÉ 2 - CKV_AWS_24 : Security Group ouvert  
resource "aws_security_group" "vulnerable_sg" {  
  name = "vulnerable-sg"  
  ingress {  
    from_port = 22  
    to_port   = 22  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"] # SSH ouvert  
  }  
}
```



```

    }
    ingress {
        from_port    = 3389
        to_port      = 3389
        protocol      = "tcp"
        cidr_blocks   = ["0.0.0.0/0"] # RDP ouvert
    }
    ingress {
        from_port    = 0
        to_port      = 65535
        protocol      = "tcp"
        cidr_blocks   = ["0.0.0.0/0"] # Tous ports
    }
    egress {
        from_port    = 0
        to_port      = 0
        protocol      = "-1"
        cidr_blocks   = ["0.0.0.0/0"]
    }
}

# VULNÉRABILITÉ 3 - CKV_AWS_16 + CKV_AWS_17 : RDS non sécurisé
resource "aws_db_instance" "vulnerable_rds" {
    engine           = "mysql"
    engine_version   = "8.0"
    instance_class    = "db.t3.micro"
    allocated_storage = 20
    username          = "admin"
    password          = "SuperPassword123!" # DANGER
    storage_encrypted = false               # VULN CKV_AWS_16
    publicly_accessible = true              # VULN CKV_AWS_17
    backup_retention_period = 0             # VULN CKV_AWS_129
    deletion_protection = false            # VULN CKV_AWS_133
    multi_az           = false
    skip_final_snapshot = true
}

# VULNÉRABILITÉ 4 - CKV_AWS_62 : IAM trop permissif
resource "aws_iam_policy" "vulnerable_policy" {
    name = "VulnerableAdminPolicy"
    policy = jsonencode({
        Version = "2012-10-17"
        Statement = [{
            Effect = "Allow"
            Action = "*" # DANGER
            Resource = "*" # DANGER
        }]
    })
}

resource "aws_iam_user" "vulnerable_user" {
    name = "vulnerable-user"
}

resource "aws_iam_user_policy_attachment" "vulnerable_attachment" {
    user = aws_iam_user.vulnerable_user.name
}

```

```
policy_arn = aws_iam_policy.vulnerable_policy.arn  
}
```

Annexe B — Code complet : secure/main.tf

```
# =====
# FICHIER SÉCURISÉ - Bonnes pratiques CIS AWS
# =====

variable "db_password" {
  description = "Mot de passe RDS"
  type        = string
  sensitive    = true # Masqué dans les logs
}

# FIX 1 - S3 privé + chiffrement + versioning
resource "aws_s3_bucket" "secure_bucket" {
  bucket = "iac-security-demo-secure-bucket"
}

resource "aws_s3_bucket_public_access_block" "pab" {
  bucket                  = aws_s3_bucket.secure_bucket.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_server_side_encryption_configuration" "sse" {
  bucket = aws_s3_bucket.secure_bucket.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.secure_bucket.id
  versioning_configuration { status = "Enabled" }
}

# FIX 2 - Security Group ports 80/443 uniquement
resource "aws_security_group" "secure_sg" {
  name        = "secure-sg"
  description = "HTTP/HTTPS uniquement depuis internet"
  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTP → redirige HTTPS"
  }
  ingress {
```

```

        from_port    = 443
        to_port      = 443
        protocol      = "tcp"
        cidr_blocks   = ["0.0.0.0/0"]
        description   = "HTTPS depuis internet"
    }
}

# FIX 3 - RDS chiffré, privé, avec backups
resource "aws_db_instance" "secure_rds" {
    engine           = "mysql"
    engine_version   = "8.0"
    instance_class    = "db.t3.micro"
    username         = "dbadmin"
    password         = var.db_password    # Variable sensible
    storage_encrypted = true              # Chiffrement activé
    publicly_accessible = false          # Non exposé
    backup_retention_period = 7           # 7 jours de backup
    deletion_protection = true           # Protection suppression
    multi_az         = true              # Haute disponibilité
}

# FIX 4 - IAM minimal privilege sur ressource précise
resource "aws_iam_policy" "secure_policy" {
    name = "SecureMinimalPolicy"
    policy = jsonencode({
        Version = "2012-10-17"
        Statement = [
            {
                Sid      = "S3ReadOnly"
                Effect    = "Allow"
                Action    = ["s3:GetObject", "s3:ListBucket"]
                Resource  = [aws_s3_bucket.secure_bucket.arn]
            },
            {
                Sid      = "CloudWatchWrite"
                Effect    = "Allow"
                Action    = [
                    "logs:CreateLogGroup",
                    "logs:CreateLogStream",
                    "logs:PutLogEvents"
                ]
                Resource  = "arn:aws:logs:eu-west-1:*:log-group:/app/*"
            }
        ]
    })
}

resource "aws_iam_role" "secure_role" {
    name = "iac-security-demo-secure-role"
    assume_role_policy = jsonencode({
        Version = "2012-10-17"
        Statement = [{
            Action    = "sts:AssumeRole"
            Effect     = "Allow"
            Principal = { Service = "ec2.amazonaws.com" }
        }]
    })
}

```

```
    ]]  
  })  
}  
  
resource "aws_iam_role_policy_attachment" "attachment" {  
  role      = aws_iam_role.secure_role.name  
  policy_arn = aws_iam_policy.secure_policy.arn  
}
```

Annexe C — Workflow GitHub Actions complet

```
name: IaC Security - Checkov SAST Scan

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read
  security-events: write

jobs:
  checkov-vulnerable:
    name: "Scan Vulnerable IaC (non-bloquant)"
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install checkov
      - run: |
          checkov -d vulnerable/ \
            --compact \
            --output json \
            --output-file-path vulnerable-report \
            || true
          checkov -d vulnerable/ --compact || true
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: checkov-vulnerable-report
          path: vulnerable-report/
          retention-days: 30

  checkov-secure:
    name: "Scan Secure IaC (bloquant)"
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install checkov
      - run: |
          checkov -d secure/ \
            --compact \
            --output json \
```

```
        --output-file-path secure-report
        checkov -d secure/ --compact
    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: checkov-secure-report
        path: secure-report/
        retention-days: 30

security-summary:
  name: "Résumé de sécurité"
  runs-on: ubuntu-latest
  needs: [checkov-vulnerable, checkov-secure]
  if: always()
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.11"
    - run: pip install checkov
    - run: |
        checkov -d vulnerable/ --compact --quiet 2>/dev/null || true
        checkov -d secure/ --compact --quiet 2>/dev/null || true
```

Annexe D — Les 35 failles détectées par Checkov

Résultats du scan `checkov -f vulnerable/main.tf --compact` :

#	ID Checkov	Ressource	Description	Sévérité
1	CKV_AWS_20	aws_s3_bucket	ACL public-read sur le bucket	CRITICAL
2	CKV_AWS_19	aws_s3_bucket	Pas de chiffrement au repos	HIGH
3	CKV_AWS_21	aws_s3_bucket	Versioning désactivé	MEDIUM
4	CKV_AWS_52	aws_s3_bucket_versioning	MFA Delete non activé	MEDIUM
5	CKV_AWS_145	aws_s3_bucket	Pas de chiffrement KMS	HIGH
6	CKV_AWS_18	aws_s3_bucket	Access logging non activé	MEDIUM
7	CKV_AWS_144	aws_s3_bucket	Réplication cross-région absente	MEDIUM
8	CKV2_AWS_6	aws_s3_bucket	Public access block absent	HIGH
9	CKV2_AWS_61	aws_s3_bucket	Pas de lifecycle configuration	LOW
10	CKV2_AWS_62	aws_s3_bucket	Pas de notifications d'événements	LOW
11	CKV_AWS_24	aws_security_group	SSH (port 22) ouvert sur 0.0.0.0/0	CRITICAL
12	CKV_AWS_25	aws_security_group	RDP (port 3389) ouvert sur 0.0.0.0/0	CRITICAL
13	CKV_AWS_23	aws_security_group	Règles SG sans description	LOW
14	CKV_AWS_382	aws_security_group	Egress tout ouvert (0.0.0.0/0 port -1)	HIGH
15	CKV2_AWS_5	aws_security_group	SG non attaché à une ressource	MEDIUM

#	ID Checkov	Ressource	Description	Sévérité
16	CKV_AWS_16	aws_db_instance	RDS storage_encrypted = false	HIGH
17	CKV_AWS_17	aws_db_instance	RDS publicly_accessible = true	CRITICAL
18	CKV_AWS_129	aws_db_instance	Backup retention = 0	MEDIUM
19	CKV_AWS_133	aws_db_instance	deletion_protection = false	MEDIUM
20	CKV_AWS_161	aws_db_instance	Pas d'authentification IAM pour RDS	MEDIUM
21	CKV_AWS_118	aws_db_instance	Enhanced Monitoring non activé	LOW
22	CKV_AWS_157	aws_db_instance	Multi-AZ désactivé	MEDIUM
23	CKV_AWS_293	aws_db_instance	Performance Insights non activé	LOW
24	CKV2_AWS_60	aws_db_instance	Copy tags to snapshots désactivé	LOW
25	CKV_AWS_62	aws_iam_policy	IAM Action = "*" (wildcard)	CRITICAL
26	CKV_AWS_355	aws_iam_policy	Resource = "*" pour actions restrictables	CRITICAL
27	CKV_AWS_290	aws_iam_policy	Write access sans contraintes	HIGH
28	CKV2_AWS_40	aws_iam_policy	Full IAM privileges	CRITICAL
29	CKV_AWS_288	aws_iam_policy	Politique admin sans MFA	HIGH
30	CKV_AWS_40	aws_iam_user_policy_attachment	Politique attachée à un utilisateur direct	MEDIUM
31	CKV_AWS_273	aws_iam_user	Accès IAM user (devrait être via SSO)	MEDIUM
32	CKV_AWS_283	aws_iam_policy	Data destruction actions sans conditions	HIGH

#	ID Checkov	Ressource	Description	Sévérité
33	CKV_AWS_289	aws_iam_policy	Actions de gestion IAM sans MFA	HIGH
34	CKV_AWS_274	aws_iam_user_policy_attachment	Utilisation de AdministratorAccess	CRITICAL
35	CKV_AWS_109	aws_iam_policy	Policy sans condition de restriction	HIGH

RÉCAPITULATIF PAR SÉVÉRITÉ

CRITICAL : 8 failles · HIGH : 11 failles · MEDIUM : 10 failles · LOW : 6 failles